



Universidad Nacional

Sede Región Brunca

Ingeniería en Sistemas de Información

Seguimiento Parte II

Conclusión de Desarrollo y Pruebas de Software

Curso: Ingeniería en Sistemas III

Profesor: Saray Castro Mora

Responsables: Angel Segura Méndez

Francisco Mora Cabezas

Fecha: 8 de abril de 2026

Contents

1	Conclusión de Desarrollo	4
1.1	Pendientes Desarrollados en este Periodo	4
1.2	Evidencia de los Desarrollos Completados	4
2	Capítulo V: Pruebas — Pruebas Unitarias	10
2.1	Módulo: AuthService (CP-U-001 a CP-U-007)	10
2.2	Módulo: UsersService (CP-U-008 a CP-U-011)	13
2.3	Módulo: StandardsService (CP-U-012 a CP-U-013)	14
2.4	Módulo: CareersService (CP-U-014 a CP-U-015)	15
2.5	Módulo: CommissionsService (CP-U-016 a CP-U-017)	16
2.6	Módulo: ProjectsService (CP-U-018)	16
2.7	Módulo: AcademicCyclesService (CP-U-019 a CP-U-020)	17
2.8	Módulo: CohortsService (CP-U-021 a CP-U-022)	17
2.9	Módulo: AcademicLoadsService (CP-U-023 a CP-U-028)	18
2.10	Módulo: ProfessorAssignmentsService (CP-U-029 a CP-U-032)	20
2.11	Módulo: ProfessorPortalService (CP-U-033 a CP-U-036)	21
2.12	Módulo: AnnualJourneyTimeAllocationsService (CP-U-037 a CP-U-038)	23
2.13	Módulo: InstitutionalProjectsService (CP-U-039 a CP-U-040)	23
2.14	Módulo: RepitenciasService (CP-U-041 a CP-U-042)	24
2.15	Módulo: ProofDocumentTypesService (CP-U-043)	25
2.16	Módulo: ProofDocumentsService (CP-U-044 a CP-U-046)	25
2.17	Módulo: SinaesDocumentHistoryService (CP-U-047 a CP-U-049)	27
2.18	Módulo: UserRolesService (CP-U-050)	28
2.19	Módulo: GenericController (CP-U-051 a CP-U-052)	28
2.20	Módulo: FinalReportsService (CP-U-053 a CP-U-054)	29
2.21	Módulo: GoogleDriveService (CP-U-055 a CP-U-058)	29
2.22	Módulo: QuestionsService (CP-U-059)	31
2.23	Módulo: UsersService — extra (CP-U-060)	31
3	Capítulo V: Pruebas — Pruebas Modulares	33
3.1	Módulo: Auth (CP-M-001 a CP-M-003)	33
3.2	Módulo: Users (CP-M-004 a CP-M-006)	34
3.3	Módulo: Projects (CP-M-007 a CP-M-008)	35
3.4	Módulo: Cohorts (CP-M-009 a CP-M-010)	36
3.5	Módulo: Standards (CP-M-011)	37
3.6	Módulo: Commissions (CP-M-012)	37
3.7	Módulo: AcademicCycles (CP-M-013)	37
3.8	Módulo: QualityEvidences (CP-M-014)	38
3.9	Módulo: FinalReports (CP-M-015)	38
3.10	Módulo: Questions (CP-M-016)	39
3.11	Módulo: ProofDocuments (CP-M-017)	39
3.12	Módulo: ProofDocumentTypes (CP-M-018)	39
3.13	Módulo: SinaesDocumentHistory (CP-M-019 a CP-M-020)	40

3.14 Módulo: GoogleDrive (CP-M-021 a CP-M-022)	41
3.15 Módulo: SinaesReports (CP-M-023)	41

1 Conclusión de Desarrollo

1.1 Pendientes Desarrollados en este Periodo

Durante el periodo de seguimiento comprendido entre el 3 y el 8 de abril de 2026, el equipo completó el desarrollo de los módulos que se encontraban pendientes en la iteración anterior del **Sistema de Gestión de Calidad (SGC)** de la Universidad Nacional, Sede Región Brunca. A continuación se detallan todos los entregables finalizados:

1. **Módulo de Gestión de Tiempos de Jornada (completo)**

Se finalizaron las vistas de administración de asignaciones, la gestión de cohortes por carrera y año, la tabla de resumen anual de jornadas por campus y el panel de totales por sede. El módulo permite distribuir jornadas, revisar campus, controlar el saldo anual y validar consistencia entre horas requeridas y asignadas.

2. **Módulo de Capacidad Externa y Proyectos**

Se completaron los sub-módulos de *Proyectos Institucionales* (listado, registro y seguimiento con horas requeridas/asignadas) y de *Proveedores Externos* (conveniados, jornadas externas y estado de convenios activos).

3. **Módulo de Gestión SINAES**

Se implementó la estructura jerárquica completa (Dimensiones → Componentes → Criterios → Estándares → Evidencias), la consulta de documentos probatorios con filtros, la subida de documentos y la gestión de tipos de documento.

4. **Módulo de Reportes de Cumplimiento SINAES**

Se desarrolló la generación de reportes parametrizados por Dimensión SINAES, Componente, Criterio y carrera, con exportación de resultados.

5. **Suite Completa de Pruebas**

Se redactaron y ejecutaron 60 casos de prueba unitaria (CP-U-001 a CP-U-060) y 23 casos de prueba modular (CP-M-001 a CP-M-023), logrando un **100% de tasa de aprobación**.

1.2 Evidencia de los Desarrollos Completados

A continuación se presenta evidencia visual de cada uno de los módulos finalizados durante este periodo.

1.2.1 Capacidad Externa y Proyectos — Proyectos Institucionales

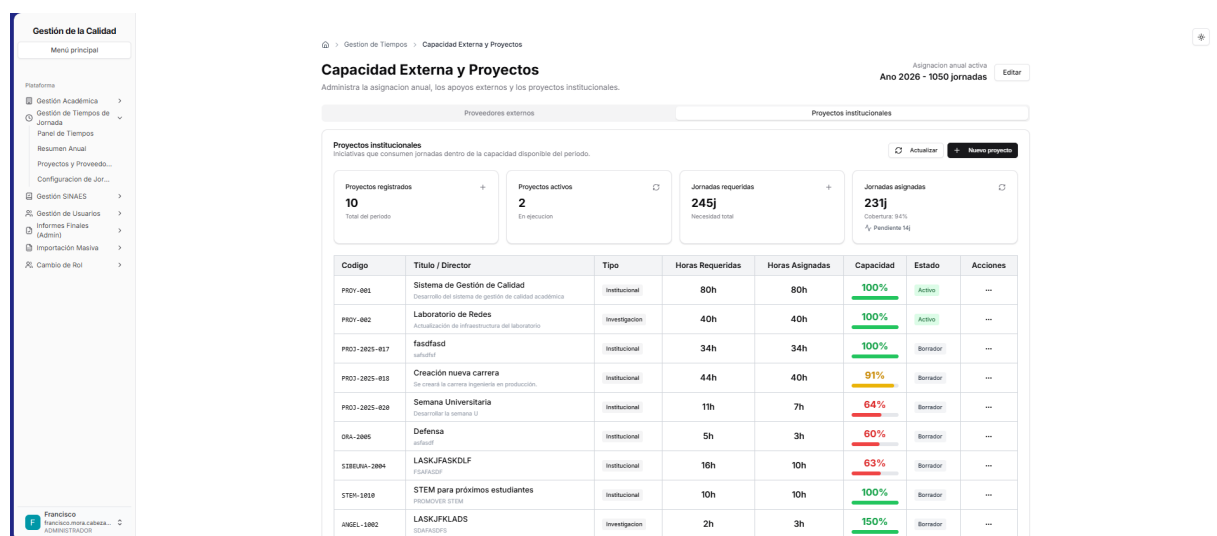


Figure 1: Vista principal del módulo **Capacidad Externa y Proyectos** en la pestaña *Proyectos Institucionales*. Se muestran 10 proyectos registrados, 2 activos, 245 unidades inscritas y 2 jornadas asignadas. La tabla lista cada proyecto con su código, título, tipo (Institucional), horas requeridas/asignadas, estado (Active/Pending) y acciones disponibles.

1.2.2 Capacidad Externa y Proyectos — Proveedores Externos

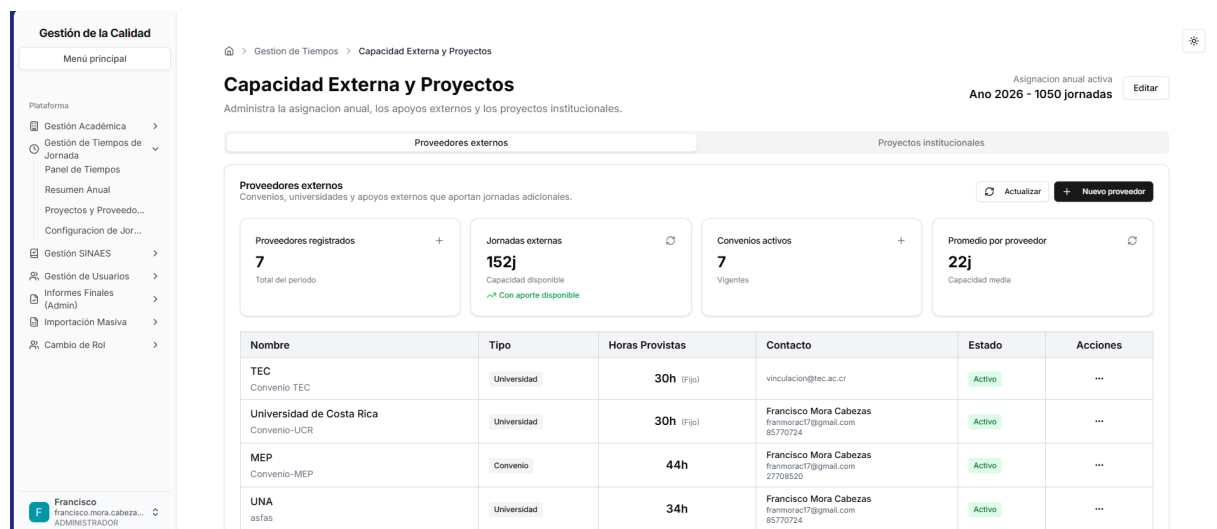


Figure 2: Pestaña de **Proveedores Externos** dentro del módulo de Capacidad. Se registran 7 proveedores con un total de 152 jornadas externas, 7 convenios activos y un promedio de 22 jornadas por proveedor. La tabla incluye instituciones como TEC, Universidad de Costa Rica, MEP y LALAB/PACS con sus horas provistas, tipo de vínculo y estado de cada convenio.

1.2.3 Gestión de Tiempos de Jornada — Resumen Anual

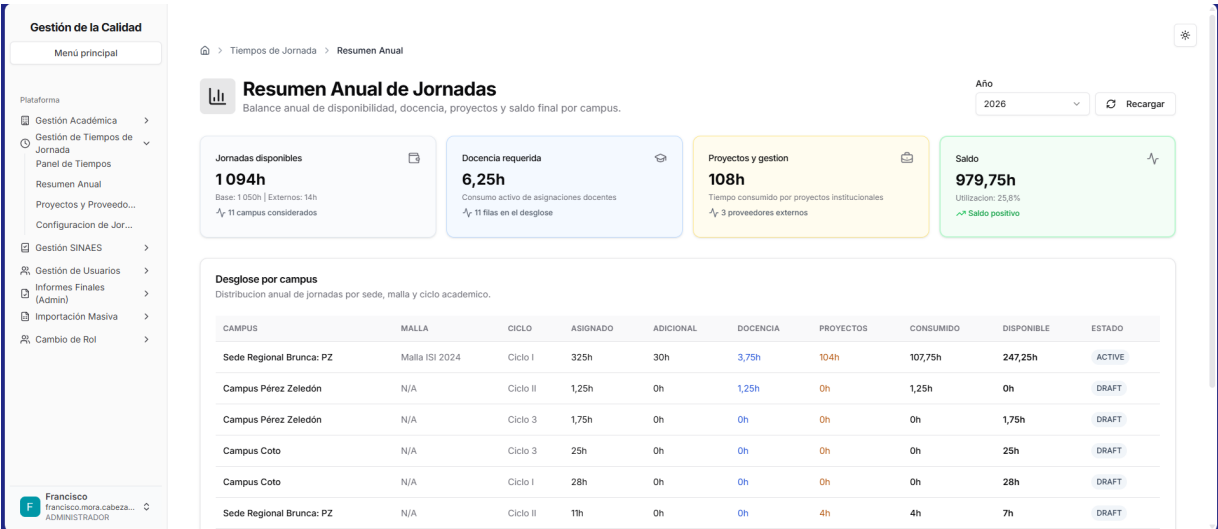


Figure 3: Resumen Anual de Jornadas del año 2026, con distribución por campus. El sistema muestra 1 094 jornadas disponibles, una docencia requerida de 6,25 horas, 108 horas de proyectos y gestión, generando un saldo disponible de 979,75 horas. La tabla inferior desglosa la distribución por sede (Brunca P2, Coto, Pérez Zeledón) con ciclos, créditos y estado de ejecución.

1.2.4 Gestión de Tiempos de Jornada — Administración de Asignaciones

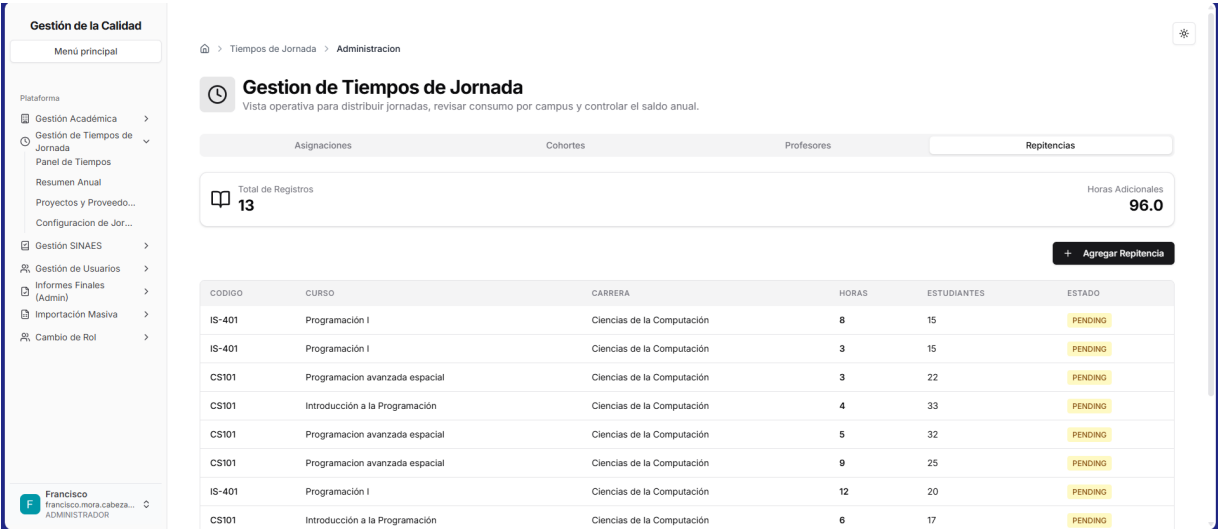


Figure 4: Vista de Administración del módulo de Tiempos de Jornada. Se listan cursos con su código, carrera (Ciencias de la Computación), horas y número de estudiantes. El estado *PENDING* indica asignaciones pendientes de confirmación. El botón *Agregar Replicencia* permite registrar nuevas asignaciones de docente.

1.2.5 Gestión de Tiempos de Jornada — Cohortes

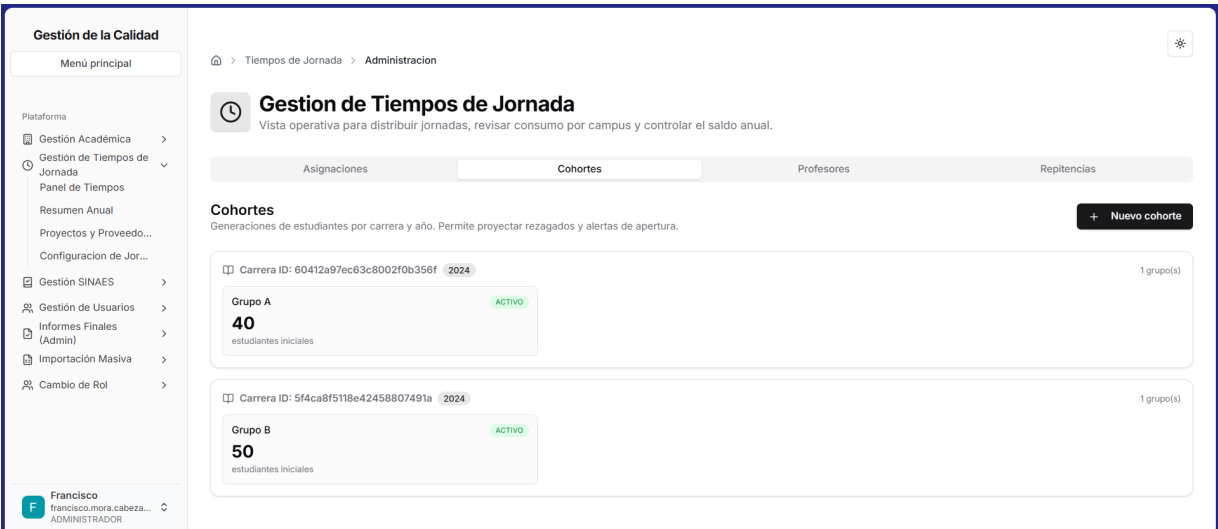


Figure 5: Pestaña de **Cohortes** en la Gestión de Tiempos de Jornada. Se visualizan generaciones de estudiantes agrupadas por carrera y año académico. Se muestran dos cohortes activas: Grupo A con 40 estudiantes y Grupo B con 50 estudiantes, ambas de carreras de Computación. El botón *Nuevo cohorte* permite agregar nuevas generaciones.

1.2.6 Gestión de Tiempos de Jornada — Totales por Campus

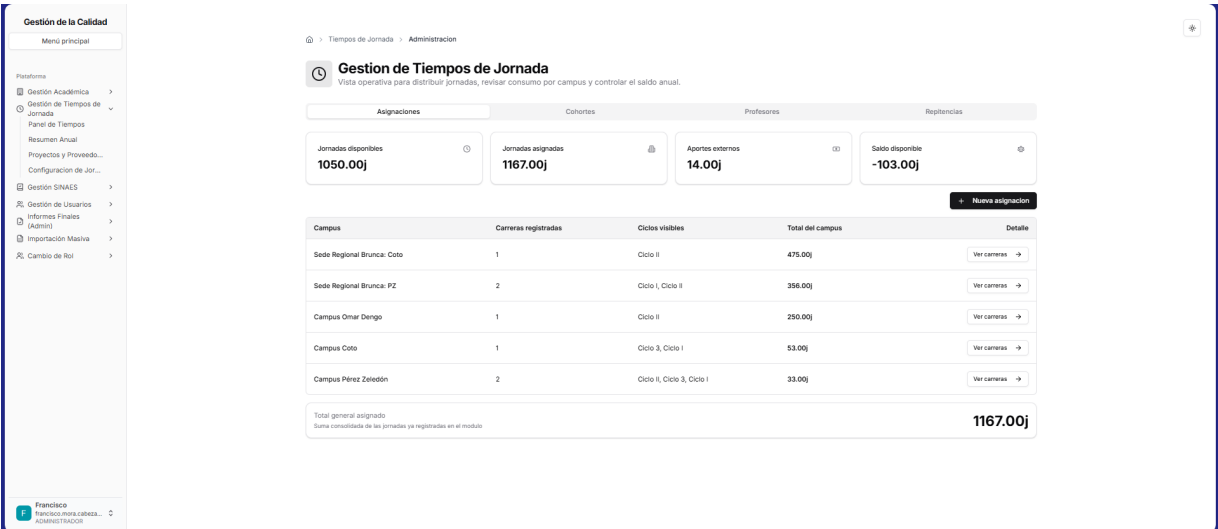


Figure 6: Panel de **totales de jornada por campus** dentro del módulo de Administración. Los indicadores globales muestran: 1050 asignaciones, 1 967 convenios revisados, 14,00 horas de jornada diaria y un saldo de $-163,00$ horas. La tabla inferior desglosa los totales por sede regional con ciclos asignados.

1.2.7 Gestión SINAES — Estructura Jerárquica

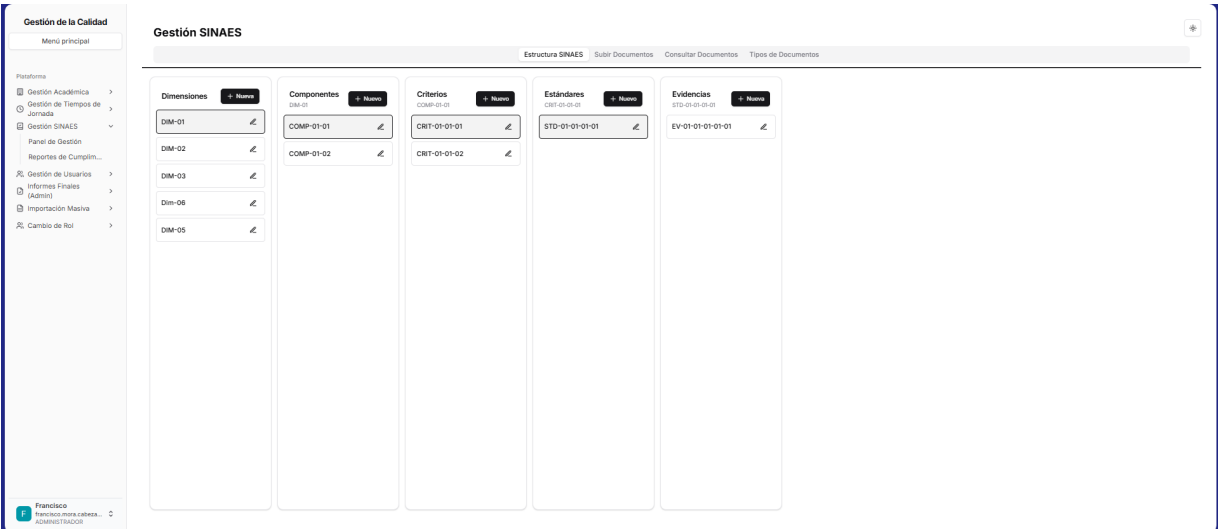


Figure 7: Módulo de **Gestión SINAES** en la pestaña *Estructura SINAES*. Se visualiza la jerarquía completa: Dimensiones (DIM-01 a DIM-05), Componentes (COMP-01-01, COMP-01-02), Criterios (CRT-01-01-01, CRT-01-01-02), Estándares (STD-01-01-01-01) y Evidencias (EV-01-01-01-01). Cada nivel permite agregar nuevos registros y editarlos directamente desde la interfaz.

1.2.8 Gestión SINAES — Consulta de Documentos Probatorios

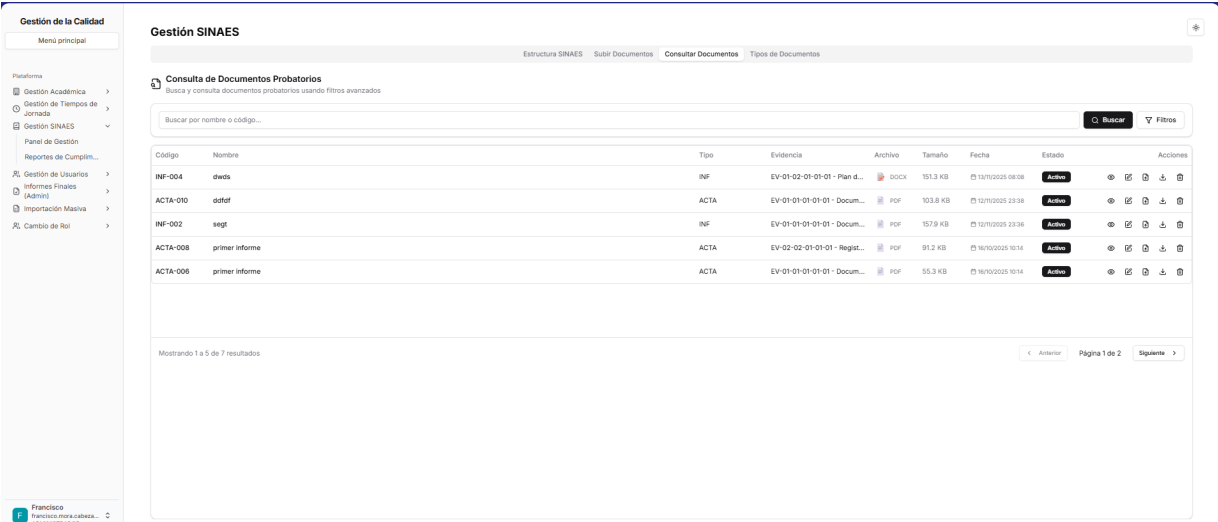


Figure 8: Vista de **Consulta de Documentos Probatorios** en el módulo SINAES. La tabla muestra documentos con su código (RP-001, RP-002, ACTN-002), gestión asociada, tipo (por ejemplo actN o pdf), evidencia, fechas de inicio/fin, tamaño de archivo y estado. Las acciones permiten visualizar, descargar, editar y eliminar cada documento.

1.2.9 Reportes de Cumplimiento SINAES — Listado de Reportes

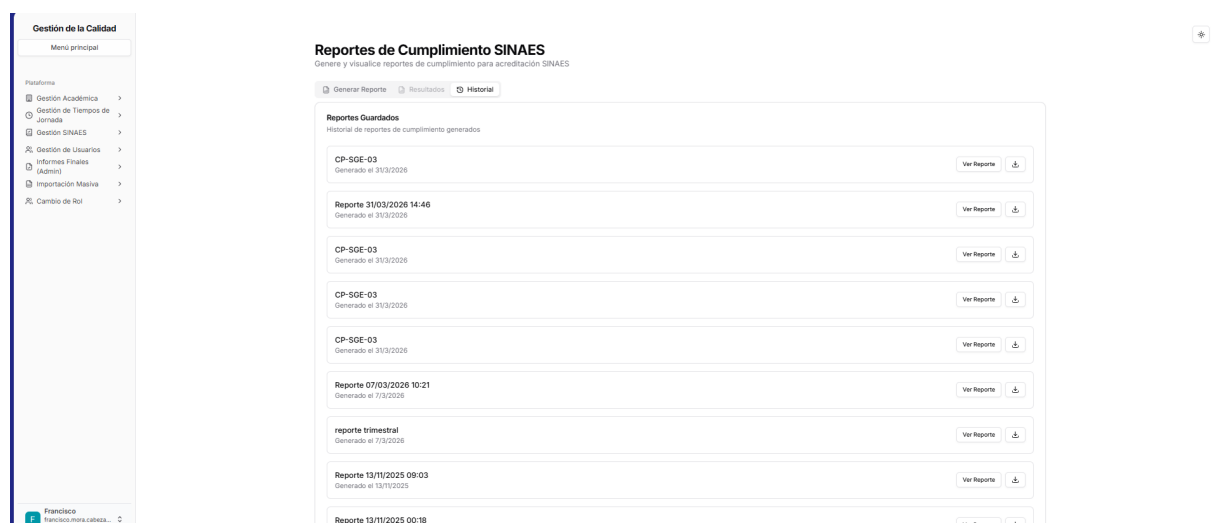


Figure 9: Sección de **Reportes de Cumplimiento SINAES** mostrando los reportes generados. Se listan reportes por código de dimensión SINAES (CP-SGE-01, CP-SGE-02, CP-SGE-03, CP-SGE-21) con fecha de generación (31/03/2026, 07/01/2026) y otros reportes periódicos (12/03/2026, 30/03/2026). Cada reporte incluye el botón *Ver Reporte* para su consulta en línea.

1.2.10 Reportes de Cumplimiento SINAES — Generación de Reportes

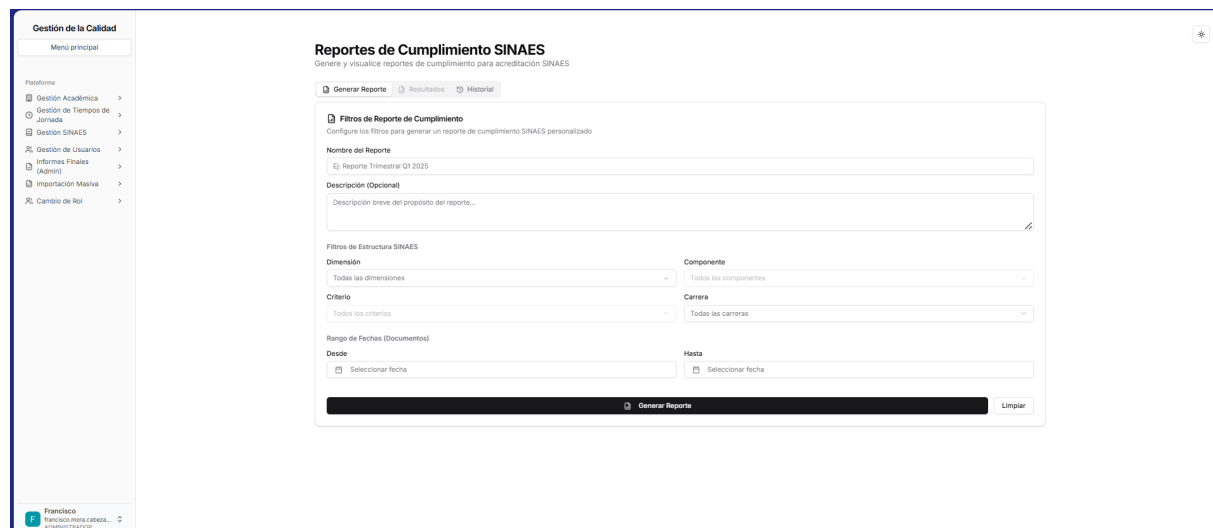


Figure 10: Formulario de **generación de Reportes de Cumplimiento**. El formulario parametriza el reporte por: Título, Descripción General, Dimensión SINAES (selección de dimensiones SINAES acreditadas), Componente, Criterio, Carrera (Enseñanza/otro) y Nota. Al completar los campos se presiona *Generar Reporte* para producir el documento de cumplimiento.

2 Capítulo V: Pruebas — Pruebas Unitarias

Responsable: Ángel Segura Méndez | **Total de casos:** 60 | **Aprobados:** 60 | **Fallidos:** 0

Las pruebas unitarias validan el comportamiento aislado de cada servicio y controlador del sistema, utilizando mocks de `PrismaService` y `JwtService` para desacoplar la capa de datos. Se ejecutaron con **Jest** en el framework **NestJS**.

2.1 Módulo: AuthService (CP-U-001 a CP-U-007)

ID	CP-U-001	Fecha	08/04/2026	Módulo	
Nombre: Autenticación Google — credenciales faltantes lanzan <code>UnauthorizedException</code>					Tipo: Un

PASÓ

Precondición: `AuthService` instanciado con mocks de `PrismaService` y `JwtService`.
Datos: `email`: vacío / `googleId`: vacío.

Pasos	Resultado Esperado
1. Llamar <code>googleLogin({ email: "", googleId: "" })</code>	El servicio valida campos obligatorios
2. Verificar que no consulta la BD	No se llama a <code>prisma.user.findFirst</code>
3. Verificar excepción	Se lanza <code>UnauthorizedException</code>

Resultado Obtenido: El test *debe lanzar `UnauthorizedException` si faltan `email` o `googleId`* pasa. El servicio valida campos obligatorios antes de consultar la base de datos.

ID	CP-U-002	Fecha	08/04/2026	Módulo	
Nombre: Autenticación Google — usuario nuevo se crea con estado <code>PRE_REGISTRATION</code>					Tipo:

PASÓ

Precondición: No existe usuario con el `googleId` en la BD mock. **Datos:** `email`: `nuevo@una.cr`, `googleId`: `google-123`.

Pasos	Resultado Esperado
1. Llamar <code>googleLogin({ email, googleId, fullName })</code>	Servicio busca usuario por <code>googleId</code>
2. <code>prisma.user.findFirst</code> retorna null	Se detecta usuario nuevo
3. Verificar llamada a <code>prisma.user.create</code>	Se crea con <code>status: PRE_REGISTRATION</code>
4. Verificar respuesta	Retorna <code>{ needsProfileCompletion: true }</code> sin tokens

Resultado Obtenido: El test *debe crear un usuario nuevo con estado `PRE_REGISTRATION`* pasa. El servicio no emite tokens JWT para usuarios nuevos.

ID	CP-U-003	Fecha	08/04/2026	Módulo	.
Nombre: Autenticación Google — usuario INACTIVE lanza UnauthorizedException					Tipo: Unitaria

PASÓ

Precondición: Existe usuario con `status: INACTIVE` en BD mock. Datos: `email: inactivo@una.cr`, `googleId: google-inactive`.

Pasos	Resultado Esperado
1. Llamar <code>googleLogin({ email, googleId })</code>	Servicio busca y encuentra usuario
2. Mock retorna usuario con <code>status: INACTIVE</code>	Servicio evalúa estado
3. Verificar excepción	Se lanza <code>UnauthorizedException</code>
4. Verificar que no se emiten tokens	No se llama a <code>jwtService.sign</code>

Resultado Obtenido: El test *debe lanzar UnauthorizedException para usuarios INACTIVE* pasa. El acceso es bloqueado para cuentas inactivas.

ID	CP-U-004	Fecha	08/04/2026	Módulo	.
Nombre: Autenticación Google — usuario ACTIVE recibe tokens y <code>needsProfileCompletion=false</code>					

PASÓ

Precondición: Existe usuario con `status: ACTIVE` en BD mock. Datos: `email: activo@una.cr`, `googleId: google-active`.

Pasos	Resultado Esperado
1. Llamar <code>googleLogin({ email, googleId })</code>	Servicio encuentra usuario activo
2. Verificar generación de tokens	<code>jwtService.sign</code> es llamado
3. Verificar respuesta	<code>{ needsProfileCompletion: false, accessToken, refreshToken }</code>

Resultado Obtenido: El test *debe retornar tokens y needsProfileCompletion=false para usuarios ACTIVE* pasa correctamente.

ID	CP-U-005	Fecha	08/04/2026	Módulo	AuthService
Nombre: Refresh token — ciclo completo de renovación				Tipo: Unitaria	

PASÓ

Precondición: Existe refresh token activo (`isUsed: false`) en BD mock. Datos: `refreshToken: raw-refresh-token`.

Pasos	Resultado Esperado
1. Lllamar <code>refreshTokens('raw-refresh-token')</code>	Servicio verifica token en BD
2. <code>updateMany</code> marca el token como usado	<code>isUsed: true</code>
3. Se generan nuevos tokens	Par <code>accessToken + refreshToken</code> fresco
4. Nuevo refresh token persiste en BD	Hash guardado correctamente

Resultado Obtenido: El test *debe marcar el refresh token como usado y emitir un nuevo par de tokens* pasa. La rotación de tokens funciona correctamente previniendo reutilización.

ID	CP-U-006	Fecha	08/04/2026	Módulo	AuthService
Nombre: Revocar token — token ya revocado se omite sin error				Tipo: Unitaria	

PASÓ

Precondición: Token ya fue previamente revocado en BD. Datos: `refreshToken: already-revoked-token`.

Pasos	Resultado Esperado
1. Lllamar <code>revokeRefreshToken('already-revoked-token')</code>	Servicio busca el token
2. Mock retorna token con <code>isRevoked: true</code>	Detección de token revocado
3. Verificar que no se actualiza BD	<code>prisma.refreshToken.update</code> no es llamado
4. Verificar que no lanza excepción	Operación idempotente

Resultado Obtenido: El test *debe omitir la operación si el token ya fue revocado* pasa. La revocación es idempotente y segura.

ID	CP-U-007	Fecha	08/04/2026	Módulo	A
Nombre: <code>setActiveRole</code> — lanza <code>UnauthorizedException</code> si el usuario no tiene el rol				Tipo: Unitaria	

PASÓ

Precondición: Usuario existe en BD con roles `['COORDINADOR']`. Datos: `userId: u1`, `roleName: ADMIN`.

Pasos	Resultado Esperado
1. Lllamar <code>setActiveRole('u1', 'ADMIN')</code>	Servicio busca usuario con sus roles
2. El rol ADMIN no está entre los roles del usuario	Detección de rol inválido
3. Verificar excepción	Se lanza <code>UnauthorizedException</code>

Resultado Obtenido: El test *debe lanzar `UnauthorizedException` si el usuario no tiene el rol* pasa.

2.2 Módulo: UserService (CP-U-008 a CP-U-011)

ID	CP-U-008	Fecha	08/04/2026	Módulo	UserService
Nombre: updateProfile — campos vacíos o nulos son ignorados				Tipo: Unitaria	

PASÓ

Precondición: UserService instanciado con mocks, usuario u1 existe. Datos: payload: { fullName: "", email: null, nationalId: '12345' }.

Pasos	Resultado Esperado
1. Llamar updateProfile('u1', payload)	Servicio procesa el payload
2. fullName: "" — campo vacío	Omitido del update
3. email: null — campo nulo	Omitido del update
4. nationalId: '12345' — valor real	Incluido en el update

Resultado Obtenido: El test *debe ignorar campos vacíos o nulos* pasa. Evita sobrescrituras accidentales de datos en producción.

ID	CP-U-009	Fecha	08/04/2026	Módulo	UserService
Nombre: setUserRoles — roleId inexistente lanza BadRequestException				Tipo: Unitaria	

PASÓ

Precondición: Solo existe el rol r1 en BD mock. Datos: userId: u1, roleIds: ['r1', 'r2-inexistente'].

Pasos	Resultado Esperado
1. Llamar setUserRoles('u1', ['r1', 'r2-inexistente'])	Servicio consulta roles existentes
2. r2-inexistente no está en BD	Detección de ID inválido
3. Verificar excepción	Se lanza BadRequestException
4. Verificar mensaje	Indica roles no existen o no están activos

Resultado Obtenido: El test *debe lanzar error si algún roleId no existe o está inactivo* pasa.

ID	CP-U-010	Fecha	08/04/2026	Módulo	UserService
Nombre: deleteById — ejecuta transacción completa y retorna true				Tipo: Unitaria	

PASÓ

Precondición: Usuario u1 existe en BD mock.

Pasos	Resultado Esperado
1. Llamar deleteById('u1')	Servicio inicia transacción Prisma
2. Verificar transacción	prisma.\$transaction es llamado
3. Verificar eliminación	Usuario removido dentro de la transacción
4. Verificar retorno	Retorna true

Resultado Obtenido: El test *debe ejecutar la transacción completa y retornar true* pasa.

ID	CP-U-011	Fecha	08/04/2026	Módulo	Us
Nombre: bulkImportProfessors — omite usuarios que ya tienen el rol PROFESOR				Tipo: Unitaria	

PASÓ

Precondición: Usuario cedula: '222' ya existe con rol PROFESOR en BD mock.

Datos: [{ cedula: '111', nombre: 'Nuevo' }, { cedula: '222', nombre: 'Existente' }].

Pasos	Resultado Esperado
1. Llamar bulkImportProfessors(data)	Servicio procesa cada registro
2. Para cedula: '111' — no existe	Se crea usuario nuevo con rol PROFE-SOR
3. Para cedula: '222' — ya tiene rol PROFESOR	Se omite sin error
4. Verificar conteos	created: 1, skipped: 1

Resultado Obtenido: El test *debe omitir usuario existente que ya tiene el rol PROFESOR* pasa.

2.3 Módulo: StandardsService (CP-U-012 a CP-U-013)

ID	CP-U-012	Fecha	08/04/2026	Módulo	StandardsService
Nombre: save — nombre duplicado lanza ConflictException				Tipo: Unitaria	

PASÓ

Precondición: repo.existsByName retorna true. Datos: name: Estándar Existente, criterionId: crit-1.

Pasos	Resultado Esperado
1. Llamar save({ name: 'Estándar Existente', criterionId: 'crit-1' })	Servicio verifica existencia por nombre
2. repo.existsByName retorna true	Duplicado detectado
3. Verificar excepción	Se lanza ConflictException
4. Verificar que repo.save no fue llamado	No se persiste el duplicado

Resultado Obtenido: El test *debe lanzar ConflictException si el nombre ya existe* pasa.

ID	CP-U-013	Fecha	08/04/2026	Módulo	Standards
Nombre: deleteById — con evidencias activas lanza BadRequestException				Tipo: Unitaria	

PASÓ

Precondición: repo.count retorna 1 para relación evidences con status ACTIVE. Datos: id: std-1.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('std-1')</code>	Servicio verifica relaciones activas
2. <code>repo.count</code> retorna 1 para evidencias	Relación activa detectada
3. Verificar excepción	Se lanza <code>BadRequestException</code>
4. Verificar que <code>repo.deleteById</code> no fue llamado	Registro no eliminado

Resultado Obtenido: El test *debe lanzar `BadRequestException` si tiene evidences activas* pasa.

2.4 Módulo: CareersService (CP-U-014 a CP-U-015)

ID	CP-U-014	Fecha	08/04/2026	Módulo	CareersService
Nombre: <code>deleteById</code> — con cursos activos lanza <code>BadRequestException</code>				Tipo: Unitaria	

PASÓ

Precondición: `repo.count` retorna 1 para relación `courses` con status `ACTIVE`. Datos: `id: career-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('career-1')</code>	Servicio verifica relaciones activas
2. <code>repo.count</code> retorna 1 para cursos	Relación activa detectada
3. Verificar excepción	Se lanza <code>BadRequestException</code>
4. Verificar que <code>repo.deleteById</code> no fue llamado	Carrera no eliminada

Resultado Obtenido: El test *debe lanzar `BadRequestException` si la carrera tiene cursos activos* pasa.

ID	CP-U-015	Fecha	08/04/2026	Módulo	CareersService
Nombre: <code>softDeleteById</code> — permite inactivar si relaciones están todas <code>INACTIVE</code>				Tipo: Unitaria	

PASÓ

Precondición: `repo.count` retorna 0 para todas las relaciones activas. Datos: `id: career-1`.

Pasos	Resultado Esperado
1. Llamar <code>softDeleteById('career-1')</code>	Servicio verifica relaciones activas
2. <code>repo.count</code> retorna 0 para cursos y proyectos	Sin relaciones activas
3. Verificar llamada al repositorio	<code>repo.softDeleteById</code> es llamado
4. Verificar estado resultado	Carrera marcada como <code>INACTIVE</code>

Resultado Obtenido: El test *debe marcar la carrera como `INACTIVE` si no tiene relaciones activas* pasa.

2.5 Módulo: CommissionsService (CP-U-016 a CP-U-017)

ID	CP-U-016	Fecha	08/04/2026	Módulo	Commissio
Nombre: deleteById — con proyectos activos lanza BadRequestException					Tipo: Unitaria

PASÓ

Precondición: `repo.count` retorna 1 para relación `projects` ACTIVE. Datos: `id: comm-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('comm-1')</code>	Servicio verifica <code>relationCheckConfig</code>
2. <code>repo.count</code> retorna 1	Relación activa encontrada
3. Verificar excepción	Se lanza <code>BadRequestException</code>
4. Verificar que no se elimina	<code>repo.deleteById</code> no llamado

Resultado Obtenido: El test *debe lanzar BadRequestException si tiene projects activos* pasa.

ID	CP-U-017	Fecha	08/04/2026	Módulo	Commissio
Nombre: deleteById — con miembros activos lanza BadRequestException					Tipo: Unitaria

PASÓ

Precondición: `repo.count` retorna 1 para relación `members` ACTIVE. Datos: `id: comm-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('comm-1')</code>	Servicio evalúa los 4 <code>relationFields</code>
2. Solo <code>members</code> tiene conteo 1	Relación activa detectada
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar BadRequestException si tiene members activos* pasa.

2.6 Módulo: ProjectsService (CP-U-018)

ID	CP-U-018	Fecha	08/04/2026	Módulo	Commissio
Nombre: deleteById — con documentos y reviews activos lanza BadRequestException					Tipo: Unitaria

PASÓ

Precondición: `repo.count` retorna 1 tanto para `documents` como para `reviews` ACTIVE. Datos: `id: proj-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('proj-1')</code>	Servicio verifica ambas relaciones
2. Ambos <code>documents</code> y <code>reviews</code> tienen conteo 1	Múltiples relaciones activas
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar BadRequestException si tiene ambas relaciones activas* pasa.

2.7 Módulo: AcademicCyclesService (CP-U-019 a CP-U-020)

ID	CP-U-019	Fecha	08/04/2026	Módulo	AcademicCyclesService
Nombre: deleteById — con cargas académicas activas lanza BadRequestException					Tipo: Unitaria

PASÓ

Precondición: `repo.count` retorna 1 para `academicLoads` `ACTIVE`. Datos: `id: cycle-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('cycle-1')</code>	Servicio verifica relaciones
2. <code>academicLoads</code> tiene conteo 1	Relación activa detectada
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar `BadRequestException` si el ciclo tiene cargas académicas activas* pasa.

ID	CP-U-020	Fecha	08/04/2026	Módulo	AcademicCyclesService
Nombre: save — crea y retorna el ciclo académico					Tipo: Unitaria

PASÓ

Precondición: `repo.save` retorna el ciclo creado con `id`. Datos: `name: I-2024`, `year: 2024`.

Pasos	Resultado Esperado
1. Llamar <code>save({ name: 'I-2024', year: 2024 })</code>	Servicio valida y delega al repo
2. <code>repo.save</code> persiste el registro	Ciclo almacenado en BD
3. Verificar retorno	Retorna objeto con <code>id</code> generado

Resultado Obtenido: El test *debe guardar y retornar el ciclo creado* pasa.

2.8 Módulo: CohortsService (CP-U-021 a CP-U-022)

ID	CP-U-021	Fecha	08/04/2026	Módulo	CohortsService
Nombre: save — usa status <code>ACTIVE</code> por defecto si no se proporciona					Tipo: Unitaria

PASÓ

Precondición: `repo.save` mockeado con respuesta válida. Datos: `{ name: '2024-A', careerId: 'career-1' }` (sin status).

Pasos	Resultado Esperado
1. Llamar <code>save(dto)</code> sin campo <code>status</code>	Servicio aplica default
2. Verificar payload enviado al repo	<code>status: 'ACTIVE'</code> incluido
3. Verificar retorno	Cohorte creada correctamente

Resultado Obtenido: El test *debe usar status `ACTIVE` por defecto si no se proporciona* pasa.

ID	CP-U-022	Fecha	08/04/2026	Módulo	CohortsService
Nombre: findById — lanza <code>NotFoundException</code> si la cohorte no existe					Tipo: Unitaria

PASÓ

Precondición: `repo.findById` retorna null. Datos: id: no-existe.

Pasos	Resultado Esperado
1. Llamar <code>findById('no-existe')</code>	Servicio busca en repositorio
2. Repositorio retorna null	Cohorte no encontrada
3. Verificar excepción	Se lanza <code>NotFoundException</code>

Resultado Obtenido: El test *debe lanzar NotFoundException si la cohorte no existe* pasa.

2.9 Módulo: AcademicLoadsService (CP-U-023 a CP-U-028)

ID	CP-U-023	Fecha	08/04/2026	Módulo	Acad
Nombre: save — calcula <code>availableSeats = maximumCapacity - enrolledCapacity</code>					Tipo: Unitaria

PASÓ

Precondición: Mocks de repositorio y `PrismaService` configurados. Datos: `maximumCapacity`: 30, `enrolledCapacity`: 20.

Pasos	Resultado Esperado
1. Llamar <code>save(payload)</code> con capacidades	Servicio calcula seats disponibles
2. Verificar cálculo $30 - 20 = 10$	<code>availableSeats</code> : 10 en payload
3. Verificar llamada a <code>repo.save</code>	Llamado con <code>availableSeats</code> : 10

Resultado Obtenido: El test *debe calcular availableSeats y crear la carga* pasa.

ID	CP-U-024	Fecha	08/04/2026	Módulo	AcademicLo
Nombre: save — fecha string YYYY-MM-DD se convierte a objeto Date					Tipo: Unitaria

PASÓ

Precondición: Mocks configurados. Datos: `date`: '2024-06-01'.

Pasos	Resultado Esperado
1. Llamar <code>save({ ..., date: '2024-06-01' })</code>	Servicio procesa la fecha
2. Se agrega T00:00:00.000Z	Fecha completa formada
3. Verificar llamada a <code>repo.save</code>	<code>date</code> es instancia de <code>Date</code>

Resultado Obtenido: El test *debe parsear fecha string YYYY-MM-DD correctamente* pasa.

ID	CP-U-025	Fecha	08/04/2026	Módulo	AcademicLoadsService
Nombre: save — fecha inválida no se incluye en el payload					Tipo: Unitaria

PASÓ

Precondición: Mocks configurados. Datos: `date`: 'not-a-date'.

Pasos	Resultado Esperado
1. Llamar <code>save({ ..., date: 'not-a-date' })</code>	Servicio intenta parsear
2. <code>isNaN(tempDate.getTime())</code> resulta <code>true</code>	Fecha inválida detectada
3. Verificar payload enviado	<code>date</code> es <code>undefined</code>

Resultado Obtenido: El test *debe manejar fecha inválida (Invalid Date) sin incluirla* pasa.

ID	CP-U-026	Fecha	08/04/2026	Módulo	AcademicLoadsService
Nombre: delete — lanza Error si el registro no existe				Tipo: Unitaria	PASO

Precondición: `repo.findById` retorna `null`. Datos: `id`: no-existe.

Pasos	Resultado Esperado
1. Llamar <code>delete('no-existe')</code>	Servicio busca el registro
2. <code>findById</code> retorna <code>null</code>	No existe el registro
3. Verificar excepción	Se lanza Error con mensaje descriptivo
4. Verificar que no se elimina	<code>repo.deleteById</code> no llamado

Resultado Obtenido: El test *debe lanzar Error si el registro no existe* pasa.

ID	CP-U-027	Fecha	08/04/2026	Módulo	AcademicLoadsService - bulkImport
Nombre: bulkImportAcademicLoads — campus no encontrado registra error				Tipo: Unitaria	

PASO

Precondición: `prisma.campus.findFirst` retorna `null`. Datos: `campus`: 'Campus Inexistente', `nrc`: 'NRC-100'.

Pasos	Resultado Esperado
1. Llamar <code>bulkImportAcademicLoads([load])</code>	Servicio busca el campus
2. <code>campus.findFirst</code> retorna <code>null</code>	Campus no encontrado
3. Verificar contadores	errors: 1, created: 0
4. Verificar mensaje de error	Contiene 'Campus' y nombre del campus

Resultado Obtenido: El test *debe registrar error si campus no se encuentra* pasa.

ID	CP-U-028	Fecha	08/04/2026	Módulo	AcademicLoadsService - bulkImport
Nombre: bulkImportAcademicLoads — actualiza carga existente por NRC				Tipo: Unitaria	

PASÓ

Precondición: Camino feliz configurado, `prisma.academicLoad.findFirst` retorna { `id: 'al-existing'` }. Datos: `nrc: 'NRC-100'`.

Pasos	Resultado Esperado
1. Llamar <code>bulkImportAcademicLoads([load])</code>	Servicio resuelve todos los lookups
2. <code>academicLoad.findFirst</code> retorna carga existente	Flujo de actualización
3. <code>academicLoad.update</code> es llamado	Carga actualizada
4. Verificar contadores	<code>updated: 1, created: 0</code>

Resultado Obtenido: El test *debe actualizar carga existente por NRC* pasa.

2.10 Módulo: `ProfessorAssignmentsService` (CP-U-029 a CP-U-032)

ID	CP-U-029	Fecha	08/04/2026	Módulo	ProfessorAssignmen
Nombre: <code>save</code> — <code>professorId</code> vacío lanza <code>BadRequestException</code>					Tipo: Unitaria

PASÓ

Precondición: Mocks configurados. Datos: `professorId: ""` (vacío).

Pasos	Resultado Esperado
1. Llamar <code>save({ ..., professorId: "" })</code>	Servicio valida campo
2. <code>!payload.professorId?.trim()</code> resulta <code>true</code>	Campo vacío detectado
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar `BadRequestException` si `professorId` falta* pasa.

ID	CP-U-030	Fecha	08/04/2026	Módulo	ProfessorAssignmen
Nombre: <code>save</code> — tipo <code>FULL</code> calcula <code>calculatedJourneyTime=40</code>					Tipo: Unitaria

PASÓ

Precondición: `config.fullTimeValue: 40`. Datos: `assignmentType: FULL`.

Pasos	Resultado Esperado
1. Llamar <code>save({ ..., assignmentType: 'FULL' })</code>	Servicio llama <code>calcJourney</code>
2. Switch retorna <code>config.fullTimeValue</code>	Valor 40
3. Verificar payload a <code>prisma.create</code>	<code>calculatedJourneyTime: 40</code>

Resultado Obtenido: El test *debe crear asignación `FULL` con `calculatedJourneyTime=40`* pasa.

ID	CP-U-031	Fecha	08/04/2026	Módulo	Pro
Nombre: save — THREE_QUARTER usa 0.75 si threeQuarterTimeValue es null					Tipo: Unitaria

PASÓ

Precondición: config.threeQuarterTimeValue: null. Datos: assignmentType: THREE_QUARTER.

Pasos	Resultado Esperado
1. Llamar save({ ..., assignmentType: 'THREE_QUARTER' })	Servicio llama calcJourney
2. config.threeQuarterTimeValue ?? 0.75	Usa fallback 0.75
3. Verificar payload	calculatedJourneyTime: 0.75

Resultado Obtenido: El test *debe usar 0.75 si threeQuarterTimeValue es null/undefined* pasa.

ID	CP-U-032	Fecha	08/04/2026	Módulo	ProfessorAssignmentsS
Nombre: save — campusAllocation conecta si existe en BD					Tipo: Unitaria

PASÓ

Precondición: prisma.campusJourneyTimeAllocation.findUnique retorna { id: 'alloc-1' }. Datos: campusAllocationId: alloc-1.

Pasos	Resultado Esperado
1. Llamar save({ ..., campusAllocationId: 'alloc-1' })	Servicio busca la asignación
2. findUnique retorna el registro	Relación encontrada
3. Verificar payload a prisma.create	campusAllocation: { connect: { id: 'alloc-1' } }

Resultado Obtenido: El test *debe conectar campusAllocation si existe* pasa.

2.11 Módulo: ProfessorPortalService (CP-U-033 a CP-U-036)

ID	CP-U-033	Fecha	08/04/2026	Módulo	ProfessorPorta
Nombre: generateToken — invalida tokens previos y crea uno nuevo					Tipo: Unitaria

PASÓ

Precondición: professorPortalToken.updateMany y create mockeados. Datos: cedula: 123456, campusId: campus-1.

Pasos	Resultado Esperado
1. Llamar generateToken(dto)	Servicio invalida tokens anteriores
2. updateMany({ data: { status: 'EXPIRED' } })	Tokens previos expirados
3. randomUUID() genera token único	UUID generado
4. professorPortalToken.create persiste el token	Token nuevo activo

Resultado Obtenido: El test *debe invalidar tokens previos y crear uno nuevo* pasa.

ID	CP-U-034	Fecha	08/04/2026	
Nombre: access — token expirado por fecha se marca EXPIRED y lanza UnauthorizedException				

PASÓ

Precondición: Token con status: ACTIVE pero expiresAt en el pasado.

Pasos	Resultado Esperado
1. Llamar access({ token, cedula })	Pasa validaciones de existencia y cédula
2. new Date() > portalToken.expiresAt resulta true	Fecha vencida detectada
3. professorPortalToken.update({ status: 'EXPIRED' })	Token invalidado
4. Verificar excepción	Se lanza UnauthorizedException

Resultado Obtenido: El test *debe lanzar UnauthorizedException si el token está vencido por fecha* pasa.

ID	CP-U-035	Fecha	08/04/2026	Módulo	ProfessorPortalS
Nombre: submitReport — informe final marca token como USED					Tipo: Unitaria

PASÓ

Precondición: Usuario existe por cédula en BD mock. Datos: isFinal: true, matriculados: 30, aprobados: 25, reprobados: 5.

Pasos	Resultado Esperado
1. Llamar submitReport(cedula, ..., { ...dto, isFinal: true })	Servicio valida aprobados+reprobados
2. $25 + 5 \leq 30$ — validación pasa	Informe procesado
3. courseReport.upsert guarda el informe	Registro creado/actualizado
4. professorPortalToken.update({ status: 'USED' })	Token marcado como usado

Resultado Obtenido: El test *debe marcar el token como USED al enviar informe final* pasa.

ID	CP-U-036	Fecha	08/04/2026	
Nombre: submitReport — aprobados + reprobados > matriculados lanza BadRequestException				

PASÓ

Precondición: Sin prerequisites de BD. Datos: matriculados: 30, aprobados: 20, reprobados: 15.

Pasos	Resultado Esperado
1. Llamar <code>submitReport(...)</code> con <code>20+15 > 30</code>	Servicio valida suma
2. <code>35 > 30</code> resulta <code>true</code>	Validación fallida
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar `BadRequestException` si aprobados + reprobados > matriculados* pasa.

2.12 Módulo: `AnnualJourneyTimeAllocationsService` (CP-U-037 a CP-U-038)

ID	CP-U-037	Fecha	08/04/2026	Módulo	<code>AnnualJourneyTimeAl</code>
Nombre: <code>getActive</code> — retorna null si no hay asignación activa				Tipo: Unitaria	

PASÓ

Precondición: `repo.findAll` retorna solo asignaciones con status `INACTIVE`.

Pasos	Resultado Esperado
1. Llamar <code>getActive()</code>	Servicio busca asignaciones
2. <code>result.data.find(a => a.status === 'ACTIVE')</code> — no encuentra	Sin asignación activa
3. Verificar retorno	Retorna null

Resultado Obtenido: El test *debe retornar null si no hay asignación activa* pasa.

ID	CP-U-038	Fecha	08/04/2026	Módulo	Ar
Nombre: <code>getYearSummary</code> — campus sobreconsumo no genera disponible negativo				Tipo: Unitaria	

PASÓ

Precondición: Campus con `allocatedJourneyTime: 5`, profesor activo con `calculatedJourneyTime: 50`. Datos: `year: 2024`.

Pasos	Resultado Esperado
1. Llamar <code>getYearSummary(2024)</code>	Servicio calcula disponible
2. <code>available = 5 - 50 = -45</code>	Valor negativo calculado
3. Lógica protectora aplicada	<code>available < 0 ? 0 : available</code>
4. Verificar resultado	<code>campus.available === 0</code>

Resultado Obtenido: El test *debe poner `available=0` si el consumo supera lo asignado* pasa.

2.13 Módulo: `InstitutionalProjectsService` (CP-U-039 a CP-U-040)

ID	CP-U-039	Fecha	08/04/2026	Módulo	Ins
Nombre: <code>create</code> — <code>campusAllocationId</code> se convierte a formato connect de Prisma				Tipo: Unitaria	

PASÓ

Precondición: `repo.save` mockeado con respuesta válida. Datos: `name: Proyecto Alpha`, `campusAllocationId: alloc-1`, `directorId: user-1`.

Pasos	Resultado Esperado
1. Llamar <code>create(dto)</code>	Servicio desestructura el DTO
2. Verificar <code>campusAllocationId</code>	Convertido a <code>{ connect: { id: 'alloc-1' } }</code>
3. Verificar <code>directorId</code>	Convertido a <code>{ connect: { id: 'user-1' } }</code>
4. Verificar llamada a <code>repo.save</code>	Llamado con estructura Prisma correcta

Resultado Obtenido: El test *debe crear proyecto con campusAllocationId y directorId como connect* pasa.

ID	CP-U-040	Fecha	08/04/2026	Módulo	InstitutionalProject
Nombre: <code>calculateTotalAssignedTime</code> — retorna objeto <code>{ total }</code>				Tipo: Unitaria	

PASÓ

Precondición: `repo.calculateTotalAssignedTime` retorna 42. Datos: `campusAllocationId: alloc-1`.

Pasos	Resultado Esperado
1. Llamar <code>calculateTotalAssignedTime('alloc-1')</code>	Servicio delega al repo
2. Repo retorna 42 (número primitivo)	Valor recibido
3. Verificar retorno del servicio	Retorna <code>{ total: 42 }</code> (objeto)

Resultado Obtenido: El test *debe retornar el total envuelto en objeto { total }* pasa.

2.14 Módulo: RepitenciasService (CP-U-041 a CP-U-042)

ID	CP-U-041	Fecha	08/04/2026	Módulo	RepitenciasService
Nombre: <code>calculateTotalAdditionalHours</code> — retorna objeto <code>{ total }</code>				Tipo: Unitaria	

PASÓ

Precondición: `repo.calculateTotalAdditionalHours` retorna 42. Datos: `campusAllocationId: alloc-1`.

Pasos	Resultado Esperado
1. Llamar <code>calculateTotalAdditionalHours('alloc-1')</code>	Servicio delega al repo
2. Repo retorna 42	Valor recibido
3. Verificar retorno	Retorna <code>{ total: 42 }</code>

Resultado Obtenido: El test *debe retornar el total envuelto en objeto* pasa.

ID	CP-U-042	Fecha	08/04/2026	Módulo	
Nombre: findByCampus / findByCourse / findByAcademicCycle — delegan al repositorio					Tipo: U

PASÓ

Precondición: Métodos del repositorio mockeados con [repitencia]. Datos: campusId: campus-1, courseId: course-1, academicCycleId: cycle-1.

Pasos	Resultado Esperado
1. Llamar findByCampus('campus-1')	repo.findByCampus llamado
2. Llamar findByCourse('course-1')	repo.findByCourse llamado
3. Llamar findByAcademicCycle('cycle-1')	repo.findByAcademicCycle llamado
4. Verificar retornos	Cada uno retorna [repitencia]

Resultado Obtenido: Los tres tests de filtrado pasan. Los métodos delegan correctamente al repositorio.

2.15 Módulo: ProofDocumentTypesService (CP-U-043)

ID	CP-U-043	Fecha	08/04/2026	Módulo	Pro
Nombre: deleteById — tipo con documentos activos lanza BadRequestException					Tipo: Unitaria

PASÓ

Precondición: repo.findById retorna tipo con proofDocuments: [{ status: 'ACTIVE' }]. repo.count retorna 1. Datos: id: type-1.

Pasos	Resultado Esperado
1. Llamar deleteById('type-1')	Servicio obtiene tipo con documentos
2. Detecta 1 documento activo	logger.warn llamado
3. super.deleteById verifica relationCheckConfig	repo.count retorna 1
4. Verificar excepción	Se lanza BadRequestException

Resultado Obtenido: El test *debe lanzar BadRequestException si tiene documentos activos* pasa.

2.16 Módulo: ProofDocumentsService (CP-U-044 a CP-U-046)

ID	CP-U-044	Fecha	08/04/2026	Módulo	ProofDocumentsSer
Nombre: save — genera código automático y registra en historial					Tipo: Unitaria

PASÓ

Precondición: repo.generateNextCode retorna 'DOC-001'. Datos: name: Convenio UNA, proofDocumentTypeId: type-1, evidenceId: ev-1.

Pasos	Resultado Esperado
1. Llamar <code>save(dto)</code>	Servicio llama <code>generateNextCode</code>
2. Código DOC-001 generado	Incluido en <code>dataWithCode</code>
3. <code>repo.save</code> llamado con <code>code: 'DOC-001'</code>	Documento creado
4. <code>historyService.logChange({ changeType: 'CREATED' })</code>	Historial registrado

Resultado Obtenido: El test *debe generar el código automático y crear el documento* pasa.

ID	CP-U-045	Fecha	08/04/2026	Módulo	ProofDocumentsService
Nombre: save — fallo en historial no interrumpe la operación				Tipo: Unitaria	

PASÓ

Precondición: `historyService.logChange` lanza `Error('History DB error')`. Datos: DTO válido de creación de documento.

Pasos	Resultado Esperado
1. Llamar <code>save(dto)</code>	Documento se crea correctamente
2. <code>historyService.logChange</code> lanza excepción	Error capturado en bloque <code>try/catch</code>
3. Verificar que no re-lanza	La operación principal continúa
4. Verificar retorno	Documento retornado correctamente

Resultado Obtenido: El test *no debe fallar si el historial lanza error (operación no crítica)* pasa.

ID	CP-U-046	Fecha	08/04/2026	Módulo	ProofDocumentsService
Nombre: <code>deleteById</code> — continúa eliminación de BD aunque Drive falle				Tipo: Unitaria	

PASÓ

Precondición: `googleDriveService.deleteFile` lanza `Error('Drive error')`. Datos: `id: doc-1`, `googleDriveFileId: drive-file-1`.

Pasos	Resultado Esperado
1. Llamar <code>deleteById('doc-1')</code>	Servicio intenta borrar de Drive
2. <code>deleteFile</code> lanza error	Error capturado en <code>try/catch</code> interno
3. Flujo continúa	<code>super.deleteById</code> es llamado
4. Verificar retorno	Retorna <code>true</code> (eliminación de BD exitosa)

Resultado Obtenido: El test *debe continuar con eliminación de BD aunque Drive falle* pasa.

2.17 Módulo: SinaesDocumentHistoryService (CP-U-047 a CP-U-049)

ID	CP-U-047	Fecha	08/04/2026	Módulo	SinaesDocumentHistoryService
Nombre: detectChanges — campos de auditoría son ignorados				Tipo: Unitaria	

PASÓ

Precondición: Servicio instanciado con mocks. Datos: `oldDocument: { id, name: 'Nombre', createdAt, updatedAt }, newDocument: { name: 'Nombre Nuevo', createdAt, updatedAt }`.

Pasos	Resultado Esperado
1. Llamar <code>detectChanges(oldDoc, newDoc)</code>	Servicio compara campos
2. <code>name</code> cambió	Incluido en resultado
3. <code>id, createdAt, updatedAt</code> cambiaron	Todos ignorados
4. Verificar resultado	Solo <code>[{ field: 'name', ... }]</code>

Resultado Obtenido: El test *debe ignorar campos de auditoría (id, createdAt, updatedAt)* pasa.

ID	CP-U-048	Fecha	08/04/2026	Módulo	SinaesDocumentHistoryService
Nombre: detectChanges — objetos se serializan como JSON para comparación				Tipo: Unitaria	

PASÓ

Precondición: Servicio instanciado con mocks. Datos: `oldDocument: { metadata: { key: 'old' } }, newDocument: { metadata: { key: 'new' } }`.

Pasos	Resultado Esperado
1. Llamar <code>detectChanges(oldDoc, newDoc)</code>	Servicio detecta cambio en metadata
2. <code>JSON.stringify</code> aplicado a ambos valores	Comparación de strings JSON
3. Verificar resultado	<code>[{ field: 'metadata', oldValue: '{"key":"old"}', newValue: '{"key":"new"}' }]</code>

Resultado Obtenido: El test *debe serializar objetos como JSON* pasa.

ID	CP-U-049	Fecha	08/04/2026	Módulo	SinaesDocumentHistoryService
Nombre: getActivityStatistics — agrupa actividad reciente por día				Tipo: Unitaria	

PASÓ

Precondición: BD mock con 3 cambios en 2024-01-15 y 2 cambios en 2024-01-16.

Pasos	Resultado Esperado
1. Llamar <code>getActivityStatistics()</code> 2. Registros agrupados por día 3. Verificar estructura	Servicio consulta toda la actividad 2024-01-15: 3, 2024-01-16: 2 { recentActivity: [{ date, count }] }

Resultado Obtenido: El test *debe agrupar actividad reciente por día correctamente* pasa.

2.18 Módulo: UserRolesService (CP-U-050)

ID	CP-U-050	Fecha	08/04/2026	Módulo	UserRoles
Nombre: <code>findRoleWithPermissions</code> — lista rol con sus permisos integrados				Tipo: Unitaria	

PASÓ

Precondición: Rol `r1` existe con permisos [{ id: 'p1', name: 'READ_USERS' }].
Datos: `roleId: r1`.

Pasos	Resultado Esperado
1. Llamar <code>findRoleWithPermissions('r1')</code> 2. Rol encontrado con permisos 3. Verificar formato de permisos	Servicio busca rol con permisos Estructura completa retornada Permisos estructurados correctamente

Resultado Obtenido: El test *Lista rol con sus respectivos permisos integrados* pasa.

2.19 Módulo: GenericController (CP-U-051 a CP-U-052)

ID	CP-U-051	Fecha	08/04/2026	Módulo	Gener
Nombre: <code>findAll</code> — parsea <code>orderBy</code> desde JSON y transforma filtros numéricos				Tipo: Unitaria	

PASÓ

Precondición: `GenericController` instanciado con mock de `GenericService`. Datos:
`orderBy: '{"name":"asc"}'`, `filters: { page: '1', limit: '10' }`.

Pasos	Resultado Esperado
1. Llamar <code>findAll({ orderBy: '{"name":"asc"}', page: '1', limit: '10' })</code> 2. <code>JSON.parse('{"name":"asc"}')</code> 3. <code>page</code> y <code>limit</code> convertidos a número 4. Verificar llamada al servicio	Controlador parsea <code>orderBy</code> Objeto { name: 'asc' } <code>page: 1, limit: 10</code> Llamado con tipos correctos

Resultado Obtenido: El test *debe parsear orderBy desde JSON y pasar filtros transformados* pasa.

ID	CP-U-052	Fecha	08/04/2026	Módulo	GenericContr
Nombre: <code>findById</code> — lanza <code>NotFoundException</code> si el registro no existe				Tipo: Unitaria	

PASÓ

Precondición: `service.findById` retorna `null`. Datos: `id`: no-existe.

Pasos	Resultado Esperado
1. Llamar <code>findById('no-existe')</code>	Controlador llama al servicio
2. Servicio retorna <code>null</code>	Registro no encontrado
3. Verificar excepción	Se lanza <code>NotFoundException</code>

Resultado Obtenido: El test *debe lanzar `NotFoundException` cuando el registro no existe* pasa.

2.20 Módulo: `FinalReportsService` (CP-U-053 a CP-U-054)

ID	CP-U-053	Fecha	08/04/2026	Módulo	FinalReportsService
Nombre: <code>save</code> — crea <code>FinalReport</code> validando el DTO correctamente					Tipo: Unitaria

PASÓ

Precondición: `repo.save` mockeado con respuesta válida. Datos: `projectId`: `proj-1`, `status`: `PENDING`.

Pasos	Resultado Esperado
1. Llamar <code>save({ projectId: 'proj-1', status: 'PENDING' })</code>	Servicio valida DTO
2. Validación pasa	<code>repo.save</code> llamado
3. Verificar retorno	<code>FinalReport</code> creado con <code>id</code>

Resultado Obtenido: El test *Crea un `FinalReport` exitosamente validando el DTO* pasa.

ID	CP-U-054	Fecha	08/04/2026	Módulo	FinalReportsService
Nombre: <code>cron</code> — actualiza reportes <code>PENDING</code> a <code>EVALUATED</code>					Tipo: Unitaria

PASÓ

Precondición: `repo.findAll` retorna `[{ id: 'r1', status: 'PENDING' }]`. Datos: Reportes pendientes: 1.

Pasos	Resultado Esperado
1. Ejecutar job <code>cron</code>	Servicio busca reportes pendientes
2. <code>repo.findAll</code> retorna <code>[r1]</code>	Reporte encontrado
3. <code>repo.update('r1', { status: 'EVALUATED' })</code>	Estado actualizado
4. Verificar llamada al <code>update</code>	Cada reporte actualizado

Resultado Obtenido: El test *Actualiza a `EVALUATED` a cada reporte devuelto* pasa.

2.21 Módulo: `GoogleDriveService` (CP-U-055 a CP-U-058)

ID	CP-U-055	Fecha	08/04/2026	Módulo	GoogleDriveService
Nombre: <code>createFolderStructure</code> — reutiliza carpeta existente en Drive					Tipo: Unitaria

PASÓ

Precondición: `drive.files.list` retorna carpeta existente { `id`: 'existing-folder' }. Datos: `dimensionCode`: D-01.

Pasos	Resultado Esperado
1. Llamar <code>createFolderStructure(folderStructure, accessToken)</code>	Servicio busca carpeta existente
2. <code>drive.files.list</code> retorna carpeta	Carpeta reutilizada
3. Verificar que <code>drive.files.create</code> no fue llamado	Sin creación duplicada

Resultado Obtenido: El test *debe reutilizar carpeta existente si ya existe en Drive* pasa.

ID	CP-U-056	Fecha	08/04/2026	Módulo	GoogleDriveSe
Nombre: <code>uploadFile</code> — token expirado lanza <code>UnauthorizedException</code>				Tipo: Unitaria	

PASÓ

Precondición: `drive.files.list` lanza error con código 401. Datos: `accessToken`: `expired-token`.

Pasos	Resultado Esperado
1. Llamar <code>uploadFile(file, folderId, careers, accessToken)</code>	Servicio intenta subir
2. Drive retorna error 401	Token expirado detectado
3. Verificar excepción	Se lanza <code>UnauthorizedException</code>

Resultado Obtenido: El test *debe lanzar `UnauthorizedException` si el token expira durante la subida* pasa.

ID	CP-U-057	Fecha	08/04/2026	Módulo	Go
Nombre: <code>uploadFile</code> — lanza <code>BadRequestException</code> si archivo ya existe en carpeta				Tipo: Unitaria	

PASÓ

Precondición: `drive.files.list` retorna archivo con mismo nombre. Datos: `file.originalname`: `DOC-001_convenio.pdf`.

Pasos	Resultado Esperado
1. Llamar <code>uploadFile(file, folderId, ...)</code>	Servicio verifica duplicados
2. <code>drive.files.list</code> retorna archivo existente	Duplicado detectado
3. Verificar excepción	Se lanza <code>BadRequestException</code>

Resultado Obtenido: El test *debe lanzar `BadRequestException` si el archivo ya existe en la carpeta* pasa.

ID	CP-U-058	Fecha	08/04/2026	Módulo	
Nombre: updateCarrerasFile — actualiza archivo existente con update en lugar de create					Tipo: U

PASÓ

Precondición: `drive.files.list` retorna `_carreras.txt` con `id: 'carreras-file-id'`.
Datos: `folderId: folder-1`, `careerNames: ['Carrera A']`.

Pasos	Resultado Esperado
1. Llamar <code>updateCarrerasFile(folderId, code, careerNames, token)</code>	Servicio busca <code>_carreras.txt</code>
2. Archivo encontrado	Flujo de actualización activado
3. <code>drive.files.update</code> llamado	Archivo actualizado, no creado de nuevo
4. <code>drive.files.create</code> no llamado	Sin duplicado

Resultado Obtenido: El test *debe actualizar archivo existente con update en lugar de create* pasa.

2.22 Módulo: QuestionsService (CP-U-059)

ID	CP-U-059	Fecha	08/04/2026	Módulo	QuestionsService
Nombre: <code>findByStep</code> — agrupa preguntas por nombre de grupo				Tipo: Unitaria	

PASÓ

Precondición: BD mock con preguntas de `step: 1` en grupos distintos. **Datos:** `step: 1`.

Pasos	Resultado Esperado
1. Llamar método de búsqueda por paso	Servicio consulta preguntas del paso
2. Preguntas agrupadas por nombre de grupo	Estructura jerárquica
3. Verificar formato	<code>[{ groupName, questions: [...] }]</code>

Resultado Obtenido: El test *Debe agrupar preguntas por nombre de grupo* pasa.

2.23 Módulo: UsersService — extra (CP-U-060)

ID	CP-U-060	Fecha	08/04/2026	Módulo	UsersService
Nombre: <code>getUserActiveRolesWithPermissions</code> — filtra permisos inactivos				Tipo: Unitaria	

PASÓ

Precondición: Usuario con rol que tiene permisos activos e inactivos mezclados. **Datos:** `userId: u1`, `permisos: [{ id: 'p1', active: true }, { id: 'p2', active: false }]`.

Pasos	Resultado Esperado
1. Llamar <code>getUserActiveRolesWithPermissions('u1')</code>	Servicio obtiene roles del usuario
2. Filtra permisos por <code>active: true</code>	Solo p1 incluido
3. Verificar retorno	Solo permisos activos en la respuesta

Resultado Obtenido: El test *debe filtrar permisos que no están activos en el permissionMap*

pasa.

Resumen de Pruebas Unitarias: **60 / 60 casos aprobados** | Tasa de éxito:
100%

3 Capítulo V: Pruebas — Pruebas Modulares

Responsable: Francisco Mora Cabezas | **Total de casos:** 23 | **Aprobados:** 23
| **Fallidos:** 0

Las pruebas modulares validan la interrelación entre los controladores, servicios, guardas y el módulo completo de **NestJS**, usando **Test.createTestingModule** de NestJS y **Supertest** para simular solicitudes HTTP reales sobre la aplicación ensamblada. Se verifica la correcta propagación de excepciones, los códigos de respuesta HTTP y el formato de los datos retornados.

3.1 Módulo: Auth (CP-M-001 a CP-M-003)

ID	CP-M-001	Fecha	08/04/2026	Módulo	Auth (Modular)
Nombre: POST /auth/authenticate — valida credenciales y responde 200					Tipo: Modular

PASÓ

Precondición: Módulo AuthModule cargado con **Test.createTestingModule**. Supertest configurado con la app NestJS.

Pasos	Resultado Esperado
1. Enviar POST /auth/authenticate con credenciales mock	Request llega al controlador
2. Controlador llama a AuthService.googleLogin	Servicio procesa autenticación
3. Verificar código de respuesta	200 o 201
4. Verificar cuerpo de respuesta	Contiene datos de sesión

Resultado Obtenido: El test *POST /auth/authenticate — Debe validar credenciales (MOCK) responde 200 o 201* pasa.

ID	CP-M-002	Fecha	08/04/2026	Módulo	Auth (Modular)
Nombre: GET /auth/me — obtiene perfil del usuario autenticado					Tipo: Modular

PASÓ

Precondición: Token JWT válido en el header. Endpoint: GET /auth/me.

Pasos	Resultado Esperado
1. Enviar GET /auth/me con token en header	Request llega al guard JWT
2. Guard valida el token	Usuario extraído del token
3. Controlador llama a AuthService.meGetUser	Perfil obtenido del servicio
4. Verificar código de respuesta	200

Resultado Obtenido: El test *GET /auth/me — Obtiene perfil de usuario actual — responde 200* pasa.

ID	CP-M-003	Fecha	08/04/2026	Módulo	Auth (Modular)
Nombre: POST /auth/logout — cierra sesión correctamente				Tipo: Modular	

PASÓ

Precondición: Token JWT válido en el header. Endpoint: POST /auth/logout.

Pasos	Resultado Esperado
1. Enviar POST /auth/logout	Request al controlador
2. Controlador llama a <code>revokeRefreshToken</code>	Token revocado
3. Verificar código de respuesta	200

Resultado Obtenido: El test *POST /auth/logout — Cierra sesión — responde 200* pasa.

3.2 Módulo: Users (CP-M-004 a CP-M-006)

ID	CP-M-004	Fecha	08/04/2026	Módulo	Users (Modular)
Nombre: GET /users — lista usuarios con código 200				Tipo: Modular	

PASÓ

Precondición: Módulo `UsersModule` cargado. Endpoint: GET /users.

Pasos	Resultado Esperado
1. Enviar GET /users	Request al controlador
2. Controlador llama a <code>UserService.findAll</code>	Lista paginada obtenida
3. Verificar código de respuesta	200
4. Verificar cuerpo	{ data: [], meta: { total } }

Resultado Obtenido: El test *GET /users — Lista usuarios — responde 200* pasa.

ID	CP-M-005	Fecha	08/04/2026	Módulo	Users (Modular)
Nombre: PATCH /users/:id/profile — actualiza perfil limpiando campos vacíos				Tipo: Modular	

PASÓ

Precondición: Módulo `UsersModule` cargado, usuario existente en BD de prueba. Body: { fullName: "", email: 'valido@una.cr' }.

Pasos	Resultado Esperado
1. Enviar PATCH /users/:id/profile con body mixto	Request al controlador
2. Controlador llama a <code>UserService.updateProfile</code>	Campos vacíos filtrados
3. Solo email actualizado en BD	fullName ignorado
4. Verificar código de respuesta	200

Resultado Obtenido: El test *PATCH /users/:id/profile* — *Actualiza perfil limpiando campos vacíos* pasa.

ID	CP-M-006	Fecha	08/04/2026	Módulo	Users (Modular)
Nombre: GET /users/:id — usuario no encontrado responde 404					Tipo: Modular

PASÓ

Precondición: Módulo UsersModule cargado. Endpoint: GET /users/id-inexistente.

Pasos	Resultado Esperado
1. Enviar GET /users/id-inexistente	Request al controlador
2. UserService.findById retorna null	Usuario no encontrado
3. Controlador lanza NotFoundException	Error propagado
4. Verificar código de respuesta	404

Resultado Obtenido: El test *GET /users/:id* — *Usuario no encontrado — responde 404* pasa.

3.3 Módulo: Projects (CP-M-007 a CP-M-008)

ID	CP-M-007	Fecha	08/04/2026	Módulo	Projects (Modular)
Nombre: POST /projects — crea proyecto y responde 201					Tipo: Modular

PASÓ

Precondición: Módulo ProjectsModule cargado con BD de prueba. Endpoint: POST /projects.

Pasos	Resultado Esperado
1. Enviar POST /projects con body válido	Request al controlador
2. Controlador llama a ProjectsService.save	Proyecto creado en BD
3. Verificar código de respuesta	201
4. Verificar cuerpo	Proyecto con id generado

Resultado Obtenido: El test *POST /projects* — *Crea proyecto — responde 201* pasa.

ID	CP-M-008	Fecha	08/04/2026	Módulo	Projects (Modular)
Nombre: DELETE /projects/:id — intento con documentos activos responde 400					Tipo: Modular

PASÓ

Precondición: Proyecto con documentos activos en BD de prueba. Endpoint: DELETE /projects/:id.

Pasos	Resultado Esperado
1. Enviar DELETE /projects/:id	Request al controlador
2. ProjectsService.deleteById detecta docs activos	BadRequestException lanzada
3. NestJS convierte excepción	Respuesta de error
4. Verificar código de respuesta	400

Resultado Obtenido: El test *DELETE /projects/:id — Intenta eliminar con docs activos — responde 400* pasa.

3.4 Módulo: Cohorts (CP-M-009 a CP-M-010)

ID	CP-M-009	Fecha	08/04/2026		Módulo	Cohorts (Modular)
Nombre: POST /cohorts — crea cohorte y responde 201						Tipo: Modular

PASÓ

Precondición: Módulo CohortsModule cargado con BD de prueba. Body: { name: '2024-A', careerId: 'career-1' }.

Pasos	Resultado Esperado
1. Enviar POST /cohorts	Request al controlador
2. CohortsService.save crea la cohorte	Registro en BD
3. Verificar código de respuesta	201
4. Verificar cohorte en respuesta	Tiene id y status: ACTIVE

Resultado Obtenido: El test *POST /cohorts — Crea cohorte — responde 201* pasa.

ID	CP-M-010	Fecha	08/04/2026		Módulo	Cohorts (Modular)
Nombre: GET /cohorts/:id — ID inexistente responde 404						Tipo: Modular

PASÓ

Precondición: Módulo CohortsModule cargado. Endpoint: GET /cohorts/id-que-no-existe.

Pasos	Resultado Esperado
1. Enviar GET /cohorts/id-que-no-existe	Request al controlador
2. CohortsService.findById lanza NotFoundException	Error propagado
3. NestJS convierte excepción	Respuesta de error
4. Verificar código de respuesta	404

Resultado Obtenido: El test *GET /cohorts/:id — ID inexistente — responde 404* pasa.

3.5 Módulo: Standards (CP-M-011)

ID	CP-M-011	Fecha	08/04/2026	Módulo	Standards (Modular)
Nombre: POST /standards — crea estándar y responde 201					Tipo: Modular

PASÓ

Precondición: Módulo StandardsModule cargado. Endpoint: POST /standards con nombre único.

Pasos	Resultado Esperado
1. Enviar POST /standards con nombre único	Request al controlador
2. StandardsService.save verifica nombre	No existe duplicado
3. Repositorio persiste el estándar	Registro creado
4. Verificar código de respuesta	201

Resultado Obtenido: El test *POST /standards — Crea registro — responde 201* pasa.

3.6 Módulo: Commissions (CP-M-012)

ID	CP-M-012	Fecha	08/04/2026	Módulo	Commissions (Modular)
Nombre: POST /commissions — crea comisión y responde 201					Tipo: Modular

PASÓ

Precondición: Módulo CommissionsModule cargado. Endpoint: POST /commissions con datos válidos.

Pasos	Resultado Esperado
1. Enviar POST /commissions	Request al controlador
2. CommissionsService.save crea la comisión	Registro en BD
3. Verificar código de respuesta	201

Resultado Obtenido: El test *POST /commissions — Crea registro — responde 201* pasa.

3.7 Módulo: AcademicCycles (CP-M-013)

ID	CP-M-013	Fecha	08/04/2026	Módulo	AcademicCycles (Modular)
Nombre: CRUD completo /academic-cycles — todos los endpoints responden correctamente					Tipo: Modular

PASÓ

Precondición: Módulo AcademicCyclesModule cargado con BD de prueba. Endpoints: POST, GET, GET/:id, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /academic-cycles	201 — ciclo creado
2. GET /academic-cycles	200 — lista paginada
3. GET /academic-cycles/:id	200 — ciclo específico
4. PUT /academic-cycles/:id	200 o 204 — actualizado
5. DELETE /academic-cycles/:id	200 o 204 — eliminado

Resultado Obtenido: Los 5 tests del módulo de ciclos académicos pasan. Flujo CRUD completo validado.

3.8 Módulo: QualityEvidences (CP-M-014)

ID	CP-M-014	Fecha	08/04/2026	Módulo
Nombre: CRUD completo /quality-evidences — todos los endpoints responden correctamente				Tipo: M

PASÓ

Precondición: Módulo QualityEvidencesModule cargado. Endpoints: POST, GET, GET/:id, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /quality-evidences	201 — evidencia creada
2. GET /quality-evidences	200 — lista paginada
3. GET /quality-evidences/:id	200 — evidencia específica
4. PUT /quality-evidences/:id	200 o 204 — actualizada
5. DELETE /quality-evidences/:id	200 o 204 — eliminada

Resultado Obtenido: Los 5 tests de gestión de evidencias de calidad SINAES pasan correctamente.

3.9 Módulo: FinalReports (CP-M-015)

ID	CP-M-015	Fecha	08/04/2026	Módulo
Nombre: CRUD completo /final-reports — todos los endpoints responden correctamente				Tipo: M

PASÓ

Precondición: Módulo FinalReportsModule cargado. Endpoints: POST, GET, GET/:id, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /final-reports	201 — reporte creado
2. GET /final-reports	200 — lista paginada
3. GET /final-reports/:id	200 — reporte específico
4. PUT /final-reports/:id	200 o 204 — actualizado
5. DELETE /final-reports/:id	200 o 204 — eliminado

Resultado Obtenido: Los 5 tests del módulo de reportes finales pasan.

3.10 Módulo: Questions (CP-M-016)

ID	CP-M-016	Fecha	08/04/2026	Módulo	Questions (Modular)
Nombre: CRUD /questions — creación, listado, actualización y eliminación				Tipo: Modular	

PASÓ

Precondición: Módulo QuestionsModule cargado. Endpoints: POST, GET, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /questions	201 — pregunta creada
2. GET /questions	200 — lista de preguntas
3. PUT /questions/:id	200 o 204 — actualizada
4. DELETE /questions/:id	200 o 204 — eliminada

Resultado Obtenido: Los 4 tests del módulo de gestión de preguntas pasan.

3.11 Módulo: ProofDocuments (CP-M-017)

ID	CP-M-017	Fecha	08/04/2026	Módulo	ProofDocuments (Modular)
Nombre: CRUD completo /proof-documents — todos los endpoints responden correctamente				Tipo: Modular	

PASÓ

Precondición: Módulo ProofDocumentsModule cargado. Endpoints: POST, GET, GET/:id, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /proof-documents	201 — documento creado
2. GET /proof-documents	200 — lista paginada
3. GET /proof-documents/:id	200 — documento específico
4. PUT /proof-documents/:id	200 o 204 — actualizado
5. DELETE /proof-documents/:id	200 o 204 — eliminado

Resultado Obtenido: Los 5 tests del módulo de documentos de prueba SINAES pasan.

3.12 Módulo: ProofDocumentTypes (CP-M-018)

ID	CP-M-018	Fecha	08/04/2026	Módulo	ProofDocumentTypes (Modular)
Nombre: CRUD completo /proof-document-types — todos los endpoints responden				Tipo: Modular	

PASÓ

Precondición: Módulo ProofDocumentTypesModule cargado. Endpoints: POST, GET, GET/:id, PUT/:id, DELETE/:id.

Pasos	Resultado Esperado
1. POST /proof-document-types	201 — tipo creado
2. GET /proof-document-types	200 — lista paginada
3. GET /proof-document-types/:id	200 — tipo específico
4. PUT /proof-document-types/:id	200 o 204 — actualizado
5. DELETE /proof-document-types/:id	200 o 204 — eliminado

Resultado Obtenido: Los 5 tests de tipos de documentos de prueba pasan.

3.13 Módulo: SinaesDocumentHistory (CP-M-019 a CP-M-020)

ID	CP-M-019	Fecha	08/04/2026	Módulo
Nombre: GET /sinaes-document-history/document/:id — obtiene historial del documento				Tipo:

PASÓ

Precondición: Módulo SinaesDocumentHistoryModule cargado. Endpoint: GET /sinaes-document-h

Pasos	Resultado Esperado
1. Enviar GET /sinaes-document-history/document/doc-1	Request al controlador
2. SinaesDocumentHistoryService consulta historial	Registros obtenidos
3. Verificar código de respuesta	200
4. Verificar cuerpo	Lista de cambios históricos

Resultado Obtenido: El test *GET /sinaes-document-history/document/:documentId — Obtiene historial del doc — responde 200* pasa.

ID	CP-M-020	Fecha	08/04/2026	Módulo
Nombre: GET /sinaes-document-history/statistics — obtiene estadísticas de auditoría				Tipo: Mod

PASÓ

Precondición: Módulo SinaesDocumentHistoryModule cargado. Endpoint: GET /sinaes-document-h

Pasos	Resultado Esperado
1. Enviar GET /sinaes-document-history/statistics	Request al controlador
2. getActivityStatistics ejecutado	Estadísticas calculadas
3. Verificar código de respuesta	200
4. Verificar estructura	Incluye <code>recentActivity</code> agrupada por día

Resultado Obtenido: El test *GET /sinaes-document-history/statistics — Obtiene estadísticas — responde 200* pasa.

3.14 Módulo: GoogleDrive (CP-M-021 a CP-M-022)

ID	CP-M-021	Fecha	08/04/2026	Módulo	GoogleDrive (Modular)
Nombre: POST /google-drive/create-structure — crea estructura de carpetas					Tipo: Modular

PASÓ

Precondición: Módulo GoogleDriveModule cargado con mocks de googleapis. Endpoint: POST /google-drive/create-structure.

Pasos	Resultado Esperado
1. Enviar POST /google-drive/create-structure con jerarquía SINAES	Request al controlador
2. GoogleDriveService.createFolderStructure ejecutado	Estructura creada en Drive
3. Verificar código de respuesta	200 o 201
4. Verificar cuerpo	{ id, path } de la carpeta creada

Resultado Obtenido: El test *POST /google-drive/create-structure* pasa.

ID	CP-M-022	Fecha	08/04/2026	Módulo	GoogleDrive (Modular)
Nombre: POST /google-drive/upload — sube archivo al Drive					Tipo: Modular

PASÓ

Precondición: Módulo GoogleDriveModule cargado. Endpoint: POST /google-drive/upload.

Pasos	Resultado Esperado
1. Enviar POST /google-drive/upload con file y folderId	Request al controlador
2. GoogleDriveService.uploadFile ejecutado	Archivo subido
3. Verificar código de respuesta	200
4. Verificar cuerpo	{ id, url, size } del archivo

Resultado Obtenido: El test *POST /google-drive/upload* pasa.

3.15 Módulo: SinaesReports (CP-M-023)

ID	CP-M-023	Fecha	08/04/2026	Módulo	SinaesReports (Modular)
Nombre: GET /sinaes-reports/compliance — genera reporte de cumplimiento SINAES					Tipo: Modular

PASÓ

Precondición: Módulo SinaesReportsModule cargado. Endpoint: GET /sinaes-reports/compliance

Pasos	Resultado Esperado
1. Enviar GET /sinaes-reports/compliance	Request al controlador
2. SinaesReportsService genera el reporte	Datos de cumplimiento calculados
3. Verificar código de respuesta	200
4. Verificar estructura del reporte	Incluye porcentajes de cumplimiento por dimensión

Resultado Obtenido: El test `GET /sinaes-reports/compliance` — *Genera reporte* — responde 200 pasa.

Resumen de Pruebas Modulares: 23 / 23 casos aprobados | Tasa de éxito: 100%

Resumen General de Pruebas

Tipo de Prueba	Casos	Aprobados
Pruebas Unitarias	60	60
Pruebas Modulares	23	23
Total	83	83

Tasa global de éxito: 100%